Ejercicios · Sesión 1 · Callbacks

Oct 22, 2026 · @Marce

Los ejercicios 1.1, 1.2 y 1.3 se escriben en `alumnado/sesion1/ejercicios.js`, debajo de cada enunciado, y se ejecutan con `node ejercicios.js`. El 1.4 se escribe en `alumnado/sesion1/semaforo.js` y se ve abriendo `semaforo.html` en el navegador. En los ejercicios de «predice», escribe tu respuesta **antes** de ejecutar el código.

1.1 De función clásica a flecha

Reescribe cada función como función flecha, en la forma más corta posible:

```
function cuadrado(x) { return x * x; }
function saludo(nombre, hora) { return 'Buenas ' + hora + ', ' + nombre; }
function crearPunto(x, y) { return { x: x, y: y }; }
function aleatorio() { return Math.random(); }
```

1.2 Mi propio forEach

Vas a escribir tus propias versiones de `forEach` y `filter`. Recuerda la función `repetir` que hemos visto en clase:

```
const repetir = (veces, fn) => {
  for (let i = 0; i < veces; i++) {
    fn(i);
  }
};
```

Usaremos estos dos arrays (ya están en `ejercicios.js`):

```
const frutas = ['manzana', 'pera', 'uva'];
const numeros = [3, 8, 1, 12];
```

a) **`miForEach(array, callback)`**. Debe hacer lo mismo que `frutas.forEach(...)`: recorrer el array y, en cada vuelta, llamar al callback pasándole el elemento y su posición.

1. Dentro, escribe un `for` que recorra el array (`i` desde 0 hasta `array.length - 1`).
2. En cada vuelta, llama al callback con dos argumentos: el elemento (`array[i]`) y su posición (`i`).

Pista: es como el ejemplo `repetir`. No hace falta `return`, porque el callback ya hace el trabajo de imprimir.

Pruébala con esta llamada:

```
miForEach(frutas, (fruta, i) => console.log(i + ': ' + fruta));
// 0: manzana
// 1: pera
// 2: uva
```

b) `miFilter(array, condicion)`. Debe hacer lo mismo que `numeros.filter(...)`: **devolver** un array nuevo solo con los elementos que cumplen la condición. `condicion` es un callback que recibe un elemento y devuelve `true` o `false`.

1. Crea un array vacío: `const resultado = [];`
2. Recorre el array, con un `for` o con tu `miForEach`.
3. Para cada elemento, llama a `condicion(elemento)`. Si devuelve `true`, añádelo con `resultado.push(elemento)`.
4. Al final, devuelve `resultado` con `return`.

Aquí sí hace falta `return`, porque quien llama a `miFilter` quiere el array. Pruébala con:

```
console.log(miFilter(numeros, n => n > 5));         // [ 8, 12 ]
console.log(miFilter(frutas, f => f.length > 4));   // [ 'manzana' ]
```

1.3 Predice el orden

Escribe la salida antes de ejecutar:

```
console.log('1');
setTimeout(() => console.log('2'), 100);
setTimeout(() => console.log('3'), 0);
console.log('4');
[5, 6].forEach(n => console.log(n));
```

Explica con tus palabras por qué el `forEach` no espera y `setTimeout(..., 0)` sí.

1.4 Semáforo con callbacks

Trabaja en `alumnado/sesion1/semaforo.js` y abre `semaforo.html` en el navegador para ver el resultado. La página tiene un círculo gris (`<div id="semaforo">`). Crea `cambiarColor(color, ms, callback)`, que pinte el fondo del círculo y llame a `callback` pasados `ms` milisegundos. Haz que el semáforo pase de verde (3 s) a ámbar (1 s) y a rojo (3 s), y que vuelva a empezar.

- Fíjate en la forma que toma tu código. ¿Cómo quedaría con diez colores?